

AI Document Processor

AI Document Processor

Complete Instruction Manual

Version GPT-4o Next.js FastAPI PostgreSQL License

**Transform scanned documents into structured, exportable data with
AI precision**

Table of Contents

Section	Description
1. Introduction	What is AI Document Processor and who is it for
2. Quick Start	Get up and running in under 5 minutes
3. System Requirements	Hardware, software, and service prerequisites
4. Installation Guide	Step-by-step installation instructions
5. User Interface Guide	Complete walkthrough of the web interface
6. Document Processing Workflow	How to process documents from upload to export
7. Template Mode	Batch processing with unified data schemas
8. API Reference	REST API endpoints and integration examples
9. Configuration Reference	Environment variables and settings
10. Security & Access Control	Authentication and security features
11. Troubleshooting	Common issues and solutions
12. Appendix	Glossary, keyboard shortcuts, and additional resources

1. Introduction

What is AI Document Processor?

AI Document Processor is an enterprise-grade web application that leverages **GPT-4o Vision** to automatically extract structured data from scanned PDF documents. It transforms unstructured document images into clean, organized Excel spreadsheets ready for analysis, reporting, and integration with your business systems.

Key Capabilities



Feature	Description
AI-Powered Extraction	Uses GPT-4o Vision for accurate data extraction from scanned documents
Auto-Detection	Automatically identifies document types and data fields
Template Mode	Aggregate multiple documents into a unified Excel template
Real-Time Processing	Track document processing status with live updates
Excel Export	Generate professionally formatted spreadsheets with metadata
Secure Access	Optional passcode-based access control

Who Should Use This Manual?

Role	Sections to Focus On
End Users	Sections 2, 5, 6, 7
Administrators	Sections 4, 9, 10
Developers	Sections 8, 9, 12
IT Support	Sections 3, 4, 11

2. Quick Start

Goal: Process your first document in under 5 minutes

Step 1: Launch the Application

Using Docker (Recommended)

```
# Clone the repository
git clone https://github.com/your-org/ai-document-processor.git
cd ai-document-processor

# Configure environment
cp .env.example .env
# Edit .env and add your OPENAI_API_KEY

# Start all services
./start-local.sh
```

Access Points:

| Service | URL |

|-----|-----|

| Web Interface | <http://localhost:3000> |

| API Documentation | <http://localhost:8000/docs> |

Step 2: Upload Your First Document



1. Open <http://localhost:3000> in your browser
2. Drag a PDF document onto the upload zone
3. Wait for processing to complete (status updates automatically)
4. Click **Download Excel** to export results

Tip: For best results, use clear scanned documents with legible text.

3. System Requirements

Minimum Hardware Requirements

Component	Requirement
CPU	2+ cores
RAM	4 GB minimum, 8 GB recommended
Storage	10 GB free space
Network	Stable internet connection (for OpenAI API)

Software Prerequisites

Software	Version	Purpose
Docker	20.10+	Container runtime
Docker Compose	2.0+	Service orchestration
Node.js	18+	Frontend development (optional)
Python	3.11+	Backend development (optional)
Git	2.30+	Version control

Required External Services

Service	Purpose	Cost
OpenAI API	GPT-4o Vision extraction	Pay-per-use
S3-Compatible Storage	Document storage (optional)	Varies

System Dependencies (for local development)

Package	Purpose
<code>poppler-utils</code>	PDF to image conversion
<code>tesseract-ocr</code>	Fallback OCR engine
<code>libpq-dev</code>	PostgreSQL client libraries

4. Installation Guide

Option A: Docker Installation (Recommended)

This is the fastest and most reliable installation method.

```
flowchart LR
    A[Clone Repo] --> B[Configure .env]
    B --> C[Run Docker Compose]
    C --> D[Access Web UI]
```

Step 1: Clone the Repository

```
git clone https://github.com/your-org/ai-document-processor.git
cd ai-document-processor
```

Step 2: Configure Environment Variables

```
cp .env.example .env
```

Edit `.env` with your settings:

```
# Required
OPENAI_API_KEY=sk-your-api-key-here

# Optional - defaults provided
DATABASE_URL=postgresql://docuser:docpass@postgres:5432/docprocessor
REDIS_URL=redis://redis:6379/0

# Optional - S3 storage
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
S3_BUCKET_NAME=
```

Step 3: Launch Services

```
# Using the helper script
./start-local.sh

# Or directly with Docker Compose
docker-compose -f docker-compose.local.yml up --build -d
```

Step 4: Verify Installation

```
# Check running containers
docker-compose ps

# Expected output:
# NAME                STATUS
# docprocessor-frontend Up (healthy)
# docprocessor-backend Up (healthy)
# docprocessor-postgres Up (healthy)
# docprocessor-redis   Up (healthy)
```

Option B: Local Development Installation

For developers who want to modify the codebase.

Backend Setup

```
cd backend

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Run database migrations
alembic upgrade head

# Start development server
uvicorn app.main:app --reload --port 8000
```

Frontend Setup

```
cd frontend

# Install dependencies
npm install

# Start development server
npm run dev
```

Quick Development Script

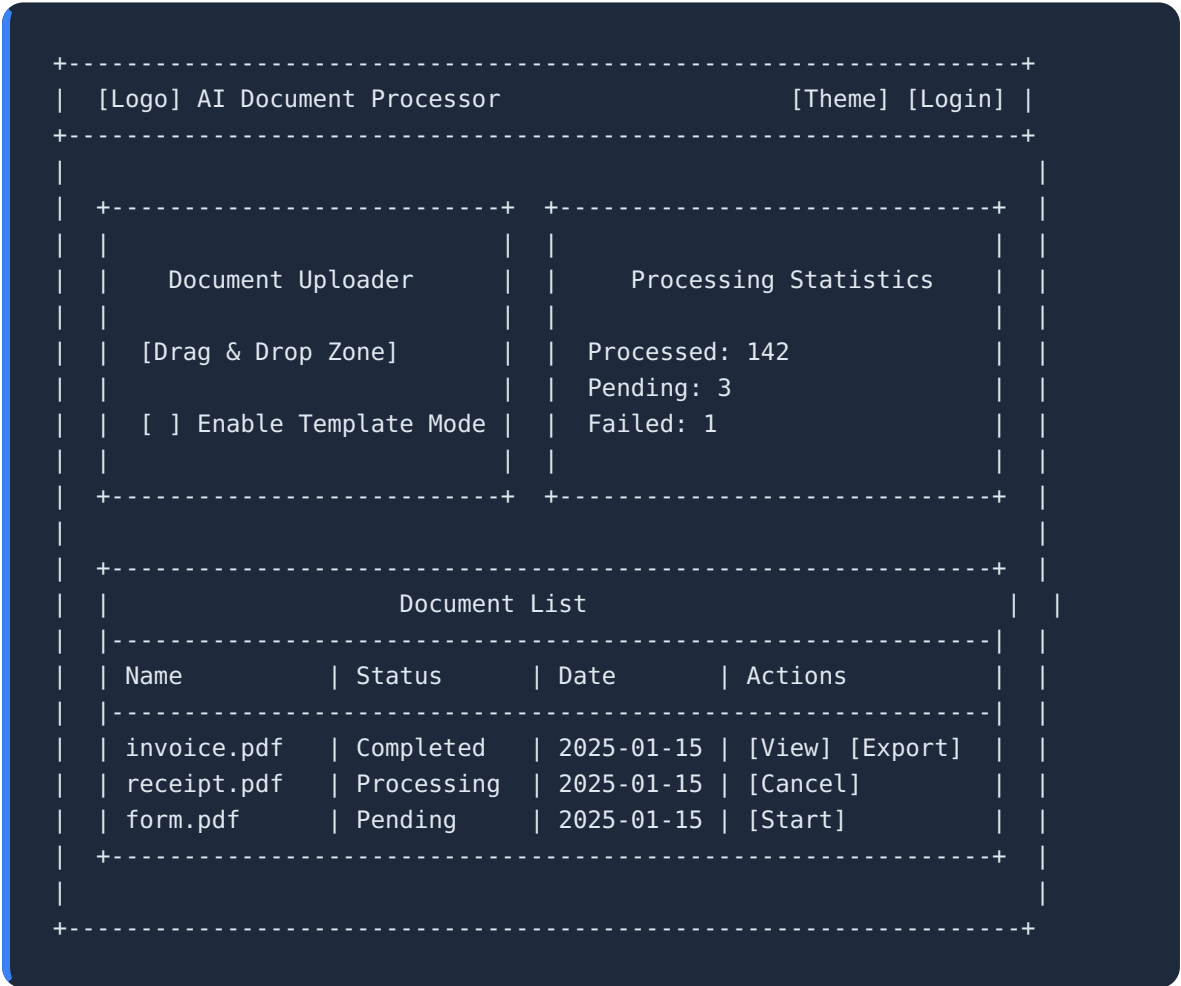
```
# Start both frontend and backend  
./scripts/dev.sh
```

Verifying Your Installation

Check	Command/Action	Expected Result
Frontend loads	Open http://localhost:3000	Upload interface displays
API responds	Open http://localhost:8000/docs	Swagger documentation loads
Database connected	Check backend logs	"Database connected successfully"
Redis connected	Check backend logs	"Redis connection established"

5. User Interface Guide

Dashboard Overview



Component Reference

Document Uploader

Element	Description
Drop Zone	Drag and drop PDF files here, or click to browse
Template Mode Toggle	Enable to aggregate multiple documents into one Excel file
File Validation	Accepts PDF files up to 50MB

Document List

Column	Description
Name	Original filename of uploaded document
Status	Current processing state (Pending, Processing, Completed, Failed)
Date	Upload timestamp
Actions	Available operations based on status

Status Indicators

Status	Icon	Description
Pending	Clock	Document queued for processing
Processing	Spinner	AI extraction in progress
Completed	Checkmark	Ready for export
Failed	X	Error occurred, check details

Theme Support

The application supports both light and dark themes:

Theme	Best For
Light	Daytime use, well-lit environments
Dark	Nighttime use, reduced eye strain

Toggle the theme using the moon/sun icon in the header.

6. Document Processing Workflow

Complete Processing Pipeline

```
sequenceDiagram
    participant U as User
    participant FE as Frontend
    participant BE as Backend
    participant AI as GPT-4o Vision
    participant DB as Database
    participant S3 as Storage

    U->>FE: Upload PDF document
    FE->>BE: POST /api/v1/documents/upload
    BE->>S3: Store original file
    BE->>DB: Create document record
    BE-->>FE: Return document ID

    Note over BE: Background Processing
    BE->>BE: Convert PDF to images
    BE->>BE: Enhance image quality
    BE->>AI: Send for extraction
    AI-->>BE: Return structured data
    BE->>DB: Save extracted data

    FE->>BE: Poll status
    BE-->>FE: Return completion
    U->>FE: Click export
    FE->>BE: GET /api/v1/documents/{id}/export/excel
    BE-->>FE: Download Excel file
```

Step-by-Step Guide

Step 1: Prepare Your Documents

Best Practices for Document Quality:

- Scan at 300 DPI or higher
- Ensure text is legible and not cut off
- Avoid skewed or rotated pages
- Use high-contrast settings when scanning

Step 2: Upload Documents

1. Navigate to the main dashboard
2. Drag your PDF file onto the upload zone
 - **Or** click the upload zone to browse files
3. Wait for the upload progress bar to complete
4. The document appears in the list with **Pending** status

Step 3: Monitor Processing

Processing happens automatically. Status updates every 5 seconds.

Phase	Duration	Description
Upload	1-5 sec	File transfer to server
Conversion	2-10 sec	PDF to image conversion
Enhancement	1-5 sec	Image quality optimization
Extraction	5-30 sec	GPT-4o Vision analysis
Finalization	1-2 sec	Data storage and indexing

Step 4: Review Extracted Data

1. Click on a **Completed** document to view details
2. Review extracted fields and values
3. Check confidence scores for each field
4. Verify data accuracy before export

Step 5: Export to Excel

1. Click the **Export** button on a completed document
2. Choose export format:
 - **Standard Export:** Single document with all extracted fields
 - **Template Export:** Multiple documents in tabular format

3. The Excel file downloads automatically

Understanding Excel Output

The exported Excel workbook contains multiple sheets:

Sheet	Contents
Extracted Data	All extracted fields and values
Metadata	Processing details, timestamps, confidence scores
Summary	Statistical overview with charts
Raw JSON	Original extraction response (for developers)

7. Template Mode

What is Template Mode?

Template Mode allows you to process multiple documents and consolidate all extracted data into a single, unified Excel spreadsheet. Each document becomes a row, and all detected fields become columns.

Standard Mode:

+-----+
| Document 1 |
| - Field A |
| - Field B |
+-----+
+-----+
| Document 2 |
| - Field A |
| - Field C |
+-----+

----->

Template Mode:

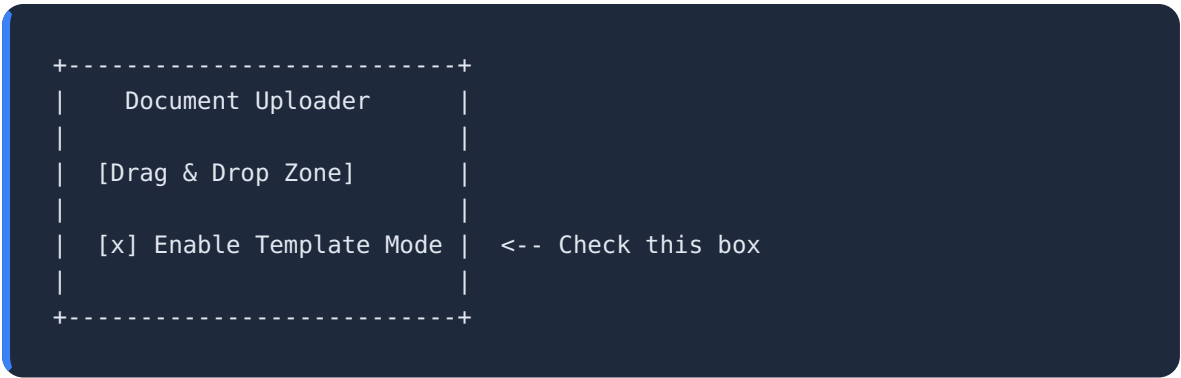
+-----+
| Field A | Field B | Field C | Field D |
+-----+
Doc 1	value	value	N/A
Doc 2	value	N/A	value
Doc 3	value	value	value
+-----+

When to Use Template Mode

Scenario	Recommended
Processing invoices from multiple vendors	Yes
Batch processing similar forms	Yes
Creating a data entry spreadsheet	Yes
Processing a single complex document	No
Documents with completely different structures	Consider

How to Use Template Mode

Step 1: Enable Template Mode



Step 2: Upload Multiple Documents

- Drag and drop multiple PDF files at once
- Or upload documents one at a time
- All documents will be associated with the template batch

Step 3: Wait for Processing

- Monitor individual document statuses
- Template export becomes available when all documents complete

Step 4: Download Template Excel

API Endpoint: GET /api/v1/documents/template/download/excel

Or use the **Download Template** button in the UI.

Template Excel Structure

Column	Description
Document Name	Original filename
Processed Date	When extraction completed
Field 1...N	All detected fields across all documents
Confidence Score	Average confidence for the row

Note: If a field is not detected in a particular document, the cell will contain "N/A".

8. API Reference

Base URL

http://localhost:8000/api/v1

Authentication

When access control is enabled:

```
Authorization: Bearer <token>
```

Core Endpoints

Documents

Method	Endpoint	Description
POST	/documents/upload	Upload a new document
GET	/documents	List all documents
GET	/documents/{id}	Get document details
GET	/documents/{id}/status	Get processing status
GET	/documents/{id}/export/excel	Download Excel export
DELETE	/documents/{id}	Delete a document

Template Mode

Method	Endpoint	Description
GET	/documents/template/download/excel	Download template Excel

Authentication

Method	Endpoint	Description
GET	/auth/status	Check authentication status
POST	/auth/login	Login with passcode
PUT	/auth/settings/login	Configure access control

Health

Method	Endpoint	Description
GET	/health	API health check

Request/Response Examples

Upload Document

Request:

```
curl -X POST "http://localhost:8000/api/v1/documents/upload" \  
-H "Content-Type: multipart/form-data" \  
-F "file=@invoice.pdf" \  
-F "template_mode=false"
```

Response:

```
{  
  "id": "doc_abc123",  
  "filename": "invoice.pdf",  
  "status": "pending",  
  "created_at": "2025-01-15T10:30:00Z"  
}
```

Get Document Status

Request:

```
curl "http://localhost:8000/api/v1/documents/doc_abc123/status"
```

Response:

```
{
  "id": "doc_abc123",
  "status": "completed",
  "progress": 100,
  "extracted_fields": 12,
  "confidence_score": 0.95,
  "completed_at": "2025-01-15T10:31:45Z"
}
```

Export to Excel

Request:

```
curl -O -J "http://localhost:8000/api/v1/documents/doc_abc123/export/excel"
```

Response: Binary Excel file download

Integration Examples

Python

```
import requests

API_BASE = "http://localhost:8000/api/v1"

# Upload document
with open("invoice.pdf", "rb") as f:
    response = requests.post(
        f"{API_BASE}/documents/upload",
        files={"file": f},
        data={"template_mode": "false"}
    )
    doc_id = response.json()["id"]

# Check status
status = requests.get(f"{API_BASE}/documents/{doc_id}/status").json()
print(f"Status: {status['status']}")

# Download Excel when complete
if status["status"] == "completed":
    excel = requests.get(f"{API_BASE}/documents/{doc_id}/export/excel")
    with open("output.xlsx", "wb") as f:
        f.write(excel.content)
```

JavaScript

```
const API_BASE = 'http://localhost:8000/api/v1';

// Upload document
const formData = new FormData();
formData.append('file', fileInput.files[0]);
formData.append('template_mode', 'false');

const uploadResponse = await fetch(`${API_BASE}/documents/upload`, {
  method: 'POST',
  body: formData
});
const { id: docId } = await uploadResponse.json();

// Check status
const statusResponse = await fetch(`${API_BASE}/documents/${docId}/status`);
const status = await statusResponse.json();

// Download Excel
if (status.status === 'completed') {
  const excelResponse = await fetch(`${API_BASE}/documents/${docId}/export/excel`);
  const blob = await excelResponse.blob();
  // Handle blob download
}
```

Error Codes

Code	Status	Description
400	Bad Request	Invalid request parameters
401	Unauthorized	Missing or invalid authentication
404	Not Found	Document not found
413	Payload Too Large	File exceeds size limit
422	Unprocessable Entity	Validation error
500	Internal Server Error	Server-side error
503	Service Unavailable	OpenAI API unavailable

9. Configuration Reference

Environment Variables

Required Variables

Variable	Description	Example
<code>OPENAI_API_KEY</code>	OpenAI API key for GPT-4o	<code>sk-...</code>
<code>DATABASE_URL</code>	PostgreSQL connection string	<code>postgresql://user:pass@host:5432/db</code>

Optional Variables

Variable	Default	Description
<code>REDIS_URL</code>	<code>redis://localhost:6379/0</code>	Redis connection URL
<code>FRONTEND_PORT</code>	<code>3000</code>	Frontend server port
<code>BACKEND_PORT</code>	<code>8000</code>	Backend API port
<code>MAX_UPLOAD_SIZE</code>	<code>52428800</code>	Max file size in bytes (50MB)
<code>PROCESSING_TIMEOUT</code>	<code>300</code>	Processing timeout in seconds

S3 Storage (Optional)

Variable	Description
<code>AWS_ACCESS_KEY_ID</code>	S3 access key
<code>AWS_SECRET_ACCESS_KEY</code>	S3 secret key
<code>S3_BUCKET_NAME</code>	Bucket name
<code>S3_ENDPOINT</code>	Custom endpoint (for MinIO, etc.)
<code>S3_REGION</code>	AWS region

Configuration Files

File	Purpose
<code>.env</code>	Environment variables (local)
<code>.env.example</code>	Environment template
<code>docker-compose.yml</code>	Production Docker config
<code>docker-compose.local.yml</code>	Local development Docker config

Network Ports

Service	Default Port	Override Variable
Frontend (Next.js)	3000	<code>FRONTEND_PORT</code>
Backend (FastAPI)	8000	<code>BACKEND_PORT</code>
PostgreSQL	5432	Via <code>DATABASE_URL</code>
Redis	6379	Via <code>REDIS_URL</code>

10. Security & Access Control

Access Control Overview

The application includes an optional passcode-based access control system.

```

flowchart TD
    A[User Visits App] --> B{Login Required?}
    B -->|No| C[Full Access]
    B -->|Yes| D[Login Prompt]
    D --> E{Valid Passcode?}
    E -->|Yes| F[Issue Token]
    F --> C
    E -->|No| G[Access Denied]
    G --> D

```

Enabling Access Control

Via Web Interface

1. Navigate to the homepage
2. Find the **Access Control** card
3. Enter a passcode (minimum 6 characters)
4. Confirm the passcode
5. Click **Enable Login**

Via API

```

curl -X PUT "http://localhost:8000/api/v1/auth/settings/login" \
  -H "Content-Type: application/json" \
  -d '{"require_login": true, "passcode": "your-secure-passcode"}'

```

Disabling Access Control

```

curl -X PUT "http://localhost:8000/api/v1/auth/settings/login" \
  -H "Content-Type: application/json" \
  -d '{"require_login": false}'

```

Security Best Practices

Practice	Recommendation
API Key Protection	Never commit <code>.env</code> to version control
HTTPS	Use HTTPS in production
Passcode Strength	Use at least 12 characters
Access Logs	Monitor <code>/docs/access-control-log.md</code>
Regular Rotation	Rotate API keys and passcodes periodically
Network Security	Restrict access to trusted networks

Data Security

Data Type	Protection
Uploaded Documents	Stored with unique IDs, optional S3 encryption
Extracted Data	Stored in PostgreSQL with access controls
API Keys	Environment variables, never logged
Session Tokens	Short-lived, secure random generation

11. Troubleshooting

Quick Diagnostics

Run this checklist when experiencing issues:

```
# Check container status
docker-compose ps

# View recent logs
docker-compose logs --tail=50

# Test API health
curl http://localhost:8000/api/v1/health

# Check database connection
docker-compose exec backend python -c "from app.db.session import engine; print('DB OK')"
```

Common Issues & Solutions

Installation Issues

Problem	Cause	Solution
Port already in use	Another service using port	Set <code>FRONTEND_PORT</code> or <code>BACKEND_PORT</code> in <code>.env</code>
Docker build fails	Missing dependencies	Run <code>docker-compose build --no-cache</code>
Permission denied	File permissions	Run <code>chmod +x start-local.sh</code>

Runtime Issues

Problem	Cause	Solution
Upload fails	File too large	Check <code>MAX_UPLOAD_SIZE</code> setting
Processing stuck	API rate limit	Wait and retry, check OpenAI quota
Export fails	Database issue	Check PostgreSQL logs
Login not working	Token expired	Clear browser storage, re-login

Processing Issues

Problem	Cause	Solution
Poor extraction quality	Low image quality	Scan at higher DPI, ensure good lighting
Missing fields	Document format not recognized	Check predefined schemas
Timeout errors	Large/complex document	Increase <code>PROCESSING_TIMEOUT</code>
OpenAI errors	API issues	Check API key, quota, network

Log Locations

Service	Access Method
Frontend	<code>docker-compose logs frontend</code>
Backend	<code>docker-compose logs backend</code>
PostgreSQL	<code>docker-compose logs postgres</code>
Redis	<code>docker-compose logs redis</code>

Getting Help

If issues persist:

1. Check the [GitHub Issues](#)
 2. Review the [Operations Manual](#)
 3. Consult the [API Documentation](#)
-

12. Appendix

A. Glossary

Term	Definition
GPT-4o Vision	OpenAI's multimodal AI model that can analyze images
OCR	Optical Character Recognition - converting images to text
Template Mode	Feature to aggregate multiple documents into one export
Confidence Score	AI's certainty about extracted data (0-1 scale)
Schema	Predefined structure for expected document fields

B. Keyboard Shortcuts

Shortcut	Action
Ctrl/Cmd + U	Open upload dialog
Ctrl/Cmd + D	Toggle dark mode
Esc	Close modal dialogs

C. Supported Document Types

The system can process various document types:

Category	Examples
Financial	Invoices, receipts, purchase orders
Legal	Contracts, agreements, forms
Medical	Lab reports, prescriptions
Administrative	Applications, registrations
Custom	Any structured document

D. File Format Support

Format	Supported	Notes
PDF	Yes	Primary supported format
JPEG/PNG	Planned	Via PDF conversion
TIFF	Planned	Via PDF conversion
Word/DOCX	No	Convert to PDF first

E. Performance Guidelines

Document Type	Expected Processing Time
Single page, clear scan	5-15 seconds
Multi-page (2-5 pages)	15-45 seconds
Complex document (10+ pages)	1-3 minutes
Batch (10 documents)	3-10 minutes

F. Version History

Version	Date	Changes
1.0.0	2025-01	Initial release
1.1.0	2025-01	Added optional login feature
1.2.0	2025-01	Template mode enhancements

Made with AI Documentation Complete

AI Document Processor

Transforming documents into data, effortlessly.

Last Updated: January 2025

For the latest documentation, visit the [GitHub Repository](#)